

1 CR Product-Company Interview Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 7 Dec 2026 – 14 Feb 2027 (Days 113–182)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm, 3:00–5:30pm, 6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import <calendar/mern-mastery-study.ics>

Read with `00–LEARNING–PATH–COMPLETE.pdf` and `19–phases–678–easy–to–hard–complete.pdf`.

This file raises the bar from generic 80 LPA prep to **~1 CR product-company** loops (Google / Meta / Amazon / Microsoft / Uber / Atlassian / Adobe / Salesforce).

Rule: Product companies hire for *problem-solving* + *product judgment* + *communication*. They do **not** hire for Kafka/K8s trivia. Infra is a tool inside a product design — never the whole round.

1. What ~1 CR actually means

Target	Typical India TC band	Level they interview you as
Google	80 L–1.5 Cr+	L4 SWE → L5 Senior
Meta	80 L–1.6 Cr+	E4 → E5
Amazon	70 L–1.2 Cr+	SDE2 → SDE3
Microsoft	70 L–1.2 Cr+	L61–L64
Uber	80 L–1.4 Cr+	L4 → L5
Atlassian	70 L–1.2 Cr+	P40–P50
Adobe / Salesforce	60 L–1 Cr+	Senior / MTS-2
Stripe / Airbnb / LinkedIn (if open)	1 Cr+	equivalent senior

6-month core (this roadmap): reach L4 / SDE2 / E4 **hire bar** (strong medium DSA + product HLD + LLD + behavioral).

Optional months 7–9 (Phase 9): push L5 / E5 / SDE3 **1 CR+ bar** (hard DSA fluency, 15+ product HLDs, company-specific loops).

You can get a 1 CR offer in 6 months if mocks pass. Use 9 months if Month 5–6 gates slip or you want L5.

2. Every question type product companies ask

Print this. A 1 CR loop is a **mix** of these — not only LeetCode.

#	Question type	Who asks it	What “pass” looks like	Where in this roadmap
1	DSA coding (1-2 medium, sometimes 1 hard)	All	Optimal or near-optimal in 30-40 min, tests, complexity	Phase 6 + daily LC
2	DSA follow-ups (stream, memory, concurrent, generalize)	Google, Meta, Uber	You change the design, not freeze	§4 below + PDF #8 §15
3	Product HLD (design Instagram / Uber / Drive)	Google, Meta, Uber, Atlassian, Adobe	Requirements → API → data → scale → tradeoffs	Phase 7 + PDF #10 + §5
4	LLD / OOP (class design, 45 min)	Amazon, Uber, Adobe, Microsoft	SOLID, extensible, no god class	Phase 6B/6D + PDF #9
5	Backend machine coding (60-90 min working code)	Amazon, Flipkart, PhonePe, Swiggy, Uber	Runnable happy path + edge cases	Phase 6G + PDF #15
6	Frontend machine coding (UI widget in 60-90 min)	Atlassian, Uber, Razorpay, Swiggy	Autocomplete / board / feed that works	Phase 6G/7G + §6
7	CS fundamentals (OS, DBMS, Networks)	Microsoft, Amazon, Adobe, some Google	Clear definitions + one real example	Phase 7G + §7
8	JS / React / Node deep dive	All fullstack / frontend roles	Event loop, hooks, rendering, Node event loop	Tiers 1-2 + theory PDFs
9	SQL / schema / indexing	Amazon, Microsoft, Adobe, Uber	Write the query + pick the index	PDF #6 + §7
10	Behavioral / LPs / Googleyness	All (Amazon heaviest)	STAR with <i>your</i> metric, not “we”	PDF #16 + §8
11	Hiring manager (scope, conflict, why us)	All	Ownership + level signal	Phase 8 + §8
12	Product sense / prioritization	Meta, Uber, Atlassian, Google (sometimes)	MVP cut, metric, tradeoff	§5 product RADIO

13	Take-home / pair programming	Some startups + a few product teams	Clean repo, tests, README	§9
14	Resume / project deep dive	All	You can defend every line of TaskFlow / ShopLite	Phase 5 + 8A
15	Compensation / team match	Recruiter + HM	Numbers + walk-away	PDF #17 Tier 5

If you only grind LC + Kafka notes, you fail types 3, 6, 7, 10, 11, 12.

3. Company loops (memorize the shape)

Google (L4 / L5)

Round	Type	Bar
Phone	1 DSA (medium-hard)	Clean code, tests, follow-up
Onsite ×4-5	2-3 DSA + 1 HLD (L5) + Googleyness	Unique-ish problems; they hate memorized solutions
Skip	Heavy Kafka/K8s unless the role is infra	Talk tradeoffs, not vendor names

DSA style: say the brute force, then optimize. They change constraints. Practice *unseen* mediums, not only tagged lists.

Meta (E4 / E5)

Round	Type	Bar
Screen	2 mediums in 45 min	Speed + communication
Onsite	2 coding + 1 product HLD + 1 behavioral	Feed / chat / typeahead common
Coding	Meta-tagged mediums	They reuse a lot – still explain, don't recast

Amazon (SDE2 / SDE3)

Round	Type	Bar
OA	2 DSA + work-simulation	Pass bar, then loop
Loop	2 DSA + 1 LLD/OOD + 2 LP-heavy	Every round has Leadership Principles
Design	Parking lot, rate limiter, locker, package router	Working code > fancy architecture

Microsoft (L61-L64)

Round	Type	Bar
Screen	1 medium DSA	Clean, compilable
Loop	2 DSA + 1 design (HLD or LLD) + HM	CS fundamentals sneak in (OS, threads, indexes)
Style	Practical, less "trick hard" than Google	Explain memory, complexity, API shape

Uber (L4 / L5)

Round	Type	Bar
Screen	DSA (graphs / heaps / intervals common)	Real-world flavor
Loop	DSA + product HLD (Uber / maps / dispatch) + sometimes frontend MC	Geospatial + matching
Extra	Code signal / take-home on some teams	

Atlassian (P40 / P50)

Round	Type	Bar
Screen	DSA medium	
Loop	Karat-style + frontend or backend MC + values	Autocomplete, board, comment UI, API design
Values	Teamwork, customer, grit	Concrete stories

Adobe / Salesforce

Round	Type	Bar
Mix	DSA medium + JS/React or Java + HLD + HM	Less "Google hard", more production sense
Salesforce	Apex rare for MERN; they still ask JS + design	Know multi-tenant / sharing at a high level

4. Product-based DSA (what they actually ask)

Generic pattern lists are necessary but not sufficient. Product companies ask a **small set of families** with **follow-ups**.

4.1 Families you must be fluent in (1 CR bar)

Family	Why product companies love it	Must-do problems
Arrays / two pointers / window	Screening + Meta speed	Two Sum, 3Sum, Container Water, Longest Substring, Min Window
Hash + prefix	Counting, anagrams, subarray sum	Group Anagrams, Subarray Sum K, Product Except Self
Intervals	Calendar, meetings, merge slots	Merge Intervals, Insert Interval, Meeting Rooms II, Non-overlap
Binary search on answer	Capacity, koko, split array	Koko, Ship Packages, Split Array Largest Sum, Median Two Arrays
Linked list	Reverse, cycle, merge	Reverse, Merge K, LRU (design), Copy Random
Trees / BST	Recursion bar	Invert, LCA, Validate BST, Serialize, Max Path Sum
Graphs BFS/DFS	Islands, rooms, word ladder	Islands, Rotting Oranges, Word Ladder, Course Schedule, Clone Graph
Heap / top-K	Feeds, scheduling	Top K, Kth Largest, Merge K, Median Stream, Task Scheduler
Union-Find	Components, accounts	Number of Provinces, Redundant Connection, Accounts Merge
Topo sort	Build systems, courses	Course Schedule I/II, Alien Dictionary
DP (1D / 2D / string / knapsack)	Google / Microsoft	House Robber, Coin Change, LIS, LCS, Edit Distance, Word Break, Unique Paths
Backtracking	Subsets, permutations, sudoku	Subsets, Permutations, Combination Sum, Word Search, N-Queens
Trie	Typeahead, word search	Implement Trie, Word Search II, Autocomplete
Monotonic stack	Next greater, histogram	Daily Temps, Largest Rectangle, Trapping Rain

Design problems	LRU, LFU, Twitter, TinyURL	LC 146, 460, 355, 535
Bit / math (light)	Occasional Google / Microsoft	Single Number, Counting Bits, Hamming

You do not need heavy segment-tree / treap for most 1 CR fullstack loops. Treat PDF #14 segment trees as **optional L5 polish** (Phase 9), not a Month 5 blocker.

4.2 Company-tagged practice (do these in Phase 6E-6F and Phase 9)

Google-flavored (think + follow-up): LC 42, 76, 124, 212, 295, 329, 394, 528, 818, 843, 1055, 1153 — plus *random mediums* from the weekly contest archive.

Meta-flavored (speed, 2 in 45 min): LC 56, 57, 215, 236, 314, 347, 398, 426, 528, 560, 670, 680, 708, 938, 1249, 1570, 1650, 1762.

Amazon-flavored (medium + LP story after): LC 1, 2, 3, 5, 15, 20, 21, 23, 42, 49, 56, 121, 127, 138, 146, 200, 238, 240, 297, 347, 767, 973, 994, 1152.

Microsoft-flavored: LC 1, 2, 4, 5, 13, 21, 33, 46, 54, 88, 103, 138, 165, 200, 206, 236, 273, 348, 415, 545.

Uber-flavored (graphs, intervals, heap): LC 23, 56, 215, 253, 295, 347, 759, 973, 994, 1091, 1197, 1345.

Practice rule: **2 timed problems + 1 follow-up spoken** (stream the input / run concurrently / use $O(1)$ extra space).

4.3 The product-company follow-up script (say this every time)

After you pass the first version:

- Correctness:** walk a failing edge case.
- Complexity:** time + space; what is the bottleneck at $n = 10^7$.
- Memory:** what if the array does not fit on one machine? (map-reduce / two-pass / external sort)
- Streaming:** what if numbers arrive one-by-one? (heap / reservoir / two pointers on stream)
- Concurrency:** two threads call this — what breaks? (locks vs immutable snapshot)
- API:** wrap it as a function a product would ship (`class Autocomplete { input(c); }`)

This follow-up skill is the difference between L4 and L5.

4.4 How to talk (they grade this)

1. Restate + constraints + 2 examples (incl. empty / duplicate)
2. Brute force + complexity (10–20 sec)
3. Better idea + why it works + complexity
4. Code (names, not i/j soup)
5. Trace example
6. Invite follow-up

Silence > 20 seconds is a no-hire signal. Narrate.

5. Product-based system design (not infra class)

5.1 Product RADIO (use this, not “I will put Kafka here”)

0. Product (3 min)
Who is the user? What is the ONE metric? What is MVP vs v2?
Example: Instagram feed metric = p99 feed load < 200ms, not “use Cassandra”.
1. Requirements (5 min)
Functional: 3–5 verbs (post, follow, fetch feed)
Non-functional: QPS, storage, latency, consistency *per use case*
Out of scope: live video, ads (unless they push)
2. API + events (5 min)
REST/RPC + payload. Write the JSON. This is product thinking.
3. Data model (7 min)
Tables / documents + keys + indexes. What is the primary key for the hot path?
4. Happy-path architecture (8 min)
Client → CDN/LB → app → cache → DB → (queue only if async is required)
5. Scale the bottleneck (10 min)
Name the hottest QPS path. Cache? Shard? Fan-out? CQRS?
Give numbers. “10M DAU × 20 feed reads × 2.5 peak / 86400 ≈ 6K QPS”
6. Failure + product fallback (5 min)
What does the user see if Redis dies? Stale feed vs error vs empty.

Kafka / K8s appear only in step 5–6 if the product needs async or deploy talk. Do not open with them.

5.2 Product designs you must be able to draw blind

Core (already in PDF #10) — keep these: URL Shortener · News Feed · Chat · Notification · E-commerce order · Payment · Uber matching

Add these (1 CR product set) — practice in Phase 7C–7F:

Design	Product question they are really asking	Deep dive they expect
YouTube / Netflix watch	How do we serve video + recommendations?	Object storage, CDN, ABR, watch history, thumbnail pipeline
Google Drive / Dropbox	How do we sync files?	Chunking, dedup, conflict, notification of changes
Typeahead / Google Search box	How do we suggest in < 50ms?	Trie / prefix index, popularity rank, cache, typo
Web search (lite)	How do we find documents?	Crawler, inverted index, rank, shard by term
Ticketmaster / BookMyShow	How do we not double-sell seats?	Hold lock, inventory, payment race, idempotency
Google Docs	How do two people type at once?	OT vs CRDT, presence, persist ops
Web crawler	How do we fetch the internet politely?	Frontier queue, politeness, dedup URL, robots
Ad click aggregator	How do we count clicks at 1M/s?	Stream aggregate, exactly-once-ish, fraud

5.3 Mini-designs (YouTube)

MVP: upload, store, play, list my videos.

Metric: start-play p99 < 2s worldwide.

API: `POST /videos` (upload URL), `GET /videos/:id/manifest`.

Data: `videos(id, owner, status, duration)`, object store for chunks, CDN in front.

Hot path: play = CDN hit. Origin = object store. Transcode = async workers (queue).

v2: recommendations = offline job + online cache, not a JOIN on request.

Failure: if transcode lags, serve original + “processing” badge.

5.4 Mini-designs (Drive / Dropbox)

MVP: upload, download, list folder, share link.

Metric: upload reliability; conflict rate.

API: `PUT /files` (chunked), `GET /files/:id`, `POST /share`.

Data: metadata DB (name, parent, version, checksum) + chunk store.

Sync: client hashes chunks → upload missing → commit metadata version.

Conflict: last-writer-wins for MVP; v2 = keep both + notify.

Notification: watchers via queue + websocket. Not polling every file.

5.5 Mini-designs (Typeahead)

MVP: prefix → top 10 queries.

Metric: p99 < 50ms.

API: `GET /suggest?q=`.

Data: in-memory trie or prefix-partitioned index; value = query + score.

Score: frequency × recency (offline job).

Cache: Redis for hot prefixes (`a`, `in`, `how`).

Personalization (v2): blend global + user recent.

Failure: if ranker dies, serve cached popular prefixes.

5.6 Consistency cheat-sheet (product language)

Product	Consistency	Why
Bank ledger / seat hold / payment	Strong	Wrong money / double seat = incident
Like count, view count	Eventual	User tolerates +1 delay
Feed	Read-your-writes for <i>my</i> post; eventual for others	Celebrity fan-out
Chat delivery	At-least-once + idempotent client	Duplicates worse than 100ms delay if not de-duped
Typeahead	Best-effort	Stale suggestion is fine

6. Frontend machine coding (Atlassian / Uber / Razorpay)

Backend Splitwise is not enough. Product frontend loops ask a **working widget**.

6.1 90-minute frontend MC framework

0–10 Clarify: data shape, keyboard, empty/error, mobile?
10–20 Component tree + state (lift state up). Write types.
20–70 Happy path + one interaction (search / drag / click)
70–85 Edge: debounce, loading, empty, a11y (label, focus)
85–90 Talk: how you'd virtualize / test / a11y next

6.2 Must-build list (do 4 of 6 timed)

Widget	Companies	Must work
Autocomplete	Atlassian, Uber, Google FE	Debounce, keyboard, highlight, cache
Infinite scroll feed	Meta, Uber	IntersectionObserver, loading, no dupes
Tic-tac-toe / Connect-4	Uber, Microsoft	Win detect, reset, no extra render bugs
Kanban / Trello column	Atlassian	Drag card, persist order in state
Comment thread	Atlassian, Meta	Nested replies, collapse, input
Pixel / image grid + lightbox	Google FE	Responsive grid, focus trap

Build these in React + TypeScript. Put them in `mern-mastery/frontend-mc/`.

6.3 Autocomplete sketch (what they want to hear)

- `useDebounceValue(q, 200)`
- Abort in-flight fetch on new keystroke
- `aria-activedescendant` + arrow keys
- Cache Map prefix → results
- Empty vs error vs loading as explicit UI states

7. CS fundamentals (Microsoft / Amazon / Adobe)

You already have HTTP (PDF #10b) and databases (PDF #6). This is the **interview pack** — definitions + one product example. Study in Phase 7G (2 days) and Phase 9.

OS

Topic	Say this	Product example
Process vs thread	Process = isolated address space; threads share heap	Node is 1 thread + event loop; CPU work blocks it
Concurrency vs parallelism	Concurrent = interleaved; parallel = same time	Node concurrent I/O; video transcode = many processes
Lock / deadlock	Wait-for cycle; prevent with lock order	Seat lock + payment lock – always lock seat then payment
Memory	Stack vs heap; GC pauses	Large feed JSON in memory → page the API
Scheduling	OS picks runnable thread	Not asked deep; know “CPU bound vs I/O bound”

DBMS

Topic	Say this	Product example
Index	B+ tree on column; speeds <code>WHERE</code> / <code>JOIN</code>	<code>follows(follower_id, followee_id)</code> composite
Transaction + ACID	Atomic, consistent, isolated, durable	Checkout: decrement stock + insert order
Isolation	Read committed vs repeatable vs serializable	Seat booking: serializable or <code>SELECT FOR UPDATE</code>
Normalization vs denorm	3NF for writes; denorm for read-heavy feed	Store <code>like_count</code> on post
EXPLAIN	Show plan; look for Seq Scan on large table	Add index if filter is selective

Networks (beyond PDF #10b)

Topic	Say this
TCP vs UDP	TCP reliable ordered; UDP low latency (video chunk, game)
TLS	Encrypts HTTP → HTTPS; certs, SNI
HTTP/2 vs 1.1	Multiplex streams; less head-of-line at connection level
WebSocket vs SSE	WS bidirectional; SSE server→client only
DNS	Name → IP; TTL; CDN CNAME

10 rapid-fire Qs to drill aloud: process vs thread • deadlock conditions • B+ tree • isolation levels • leftover dirty read • TCP handshake • TLS termination at LB • idempotent PUT vs POST • CORS • cookie vs localStorage.

8. Behavioral by company (do not reuse one story blindly)

Write 10 STAR stories (PDF #16) then tag them:

Story	Amazon LP	Google	Meta	Uber
Production incident you owned	Bias for Action, Dive Deep	Ambiguity	Resilience	Ownership
Disagreed with manager	Have Backbone, Disagree & Commit	Collaboration	Conflict	Courage
Mentored someone	Hire & Develop	Community	Growth	Team
Cut scope to ship	Deliver Results	Prioritization	Impact	Bias to action
Failed estimate	Learn & Be Curious	Learning	Growth	Humility
Cross-team API fight	Earn Trust	Influence	Cross-func	Stakeholder
Performance win (p99 / cost)	Invent & Simplify	Technical depth	Impact	Metrics
Customer / user bug	Customer Obsession	User focus	Empathy	Rider/driver
Said no to a feature	Think Big + Deliver	Judgment	Prioritization	Tradeoff
On-call / reliability	Insist on Highest Standards	Quality	Craft	Safety

Amazon: **name the LP**. Google: **collaboration + ambiguity**. Meta: **impact numbers**. Uber: **marketplace / two-sided thinking**.

Hiring manager extra questions:

- Why this company / this team?
- What is the hardest bug you shipped?
- How do you handle an underperforming peer?
- Where do you want to be in 2 years? (IC, not “manager” unless true)

9. Take-home, pair, resume

Take-home (48h):

- README with how to run, assumptions, tradeoffs
- Tests for the core rule
- No extra frameworks for show

- Time-box: 6–8 focused hours, then stop

Pair programming:

- Driver/navigator; ask before typing a big change
- Tiny commits of thought: “I’ll extract this function”

Resume (1 CR signal):

- 1 page. Each bullet = verb + system + metric
Bad: “Worked on backend.”
Good: “Cut TaskFlow notification p99 from 1.2s → 180ms by moving fan-out to a queue + Redis.”
- Only list what you can defend for 20 minutes.

10. How to use this in the 6-month plan

Do **not** add a second full syllabus. Overlay this file on Phases 6–8:

When	Extra from this file
Days 113–132 (6A–6E)	Families in §4.1; add intervals + backtracking problems
Days 133–135 (6F)	Company-tagged set for your first target company
Days 136–140 (6G)	1 backend MC + 1 frontend MC (autocomplete)
Days 141–156 (7A–7D)	Product RADIO §5.1 on every design
Days 161–164 (7F)	YouTube, Drive, Typeahead, Search – not Kafka-as-the-topic
Days 165–168 (7G)	CS pack §7 + company loop map §3 + frontend MC #2
Days 169–182 (8)	Mock the loop shape of the company you applied to
After Day 182	Phase 9 below if no 1 CR offer yet

Kafka (PDF #12) and K8s (PDF #13) stay as **reference**: use them when a design needs a queue or a deploy story. They are not the 1 CR core.

11. Phase 9 – optional months 7-9 (Days 183-273)

Use this if you want L5 / E5 / SDE3 or you missed Phase 6–7 gates.

Month	Days (relative)	Focus	Output
7	183-210	Hard DSA fluency: 5 problems/week hard + Meta/Google tagged mediums daily	280+ LC, 40+ hard <i>attempted</i>
7	weekends	4 more product HLDs: Docs, Crawler, Ticketmaster, Ad aggregator	14 HLD packet
8	211-238	2 full company loops/week (record yourself) + HM stories	8 recorded loops
8		Frontend MC remaining widgets + 1 take-home style repo	frontend-mc + polished README
9	239-266	Apply 3×/week; onsite prep; compensation	Pipeline of 15 companies
9	267-273	Offer compare, team match, close	Decision

Phase 9 weekly template (26h, same slots):

- Mon–Thu: 1 timed medium + 1 tagged follow-up (2h)
- Fri: 1 hard attempt + review (2h)
- Sat: 1 product HLD blind (60 min) + 1 frontend or backend MC (90 min)
- Sun: 1 full mock loop (3h) + STAR + apply

Phase 9 gate: 3 consecutive full-loop mocks at “hire” (DSA + product HLD + behavioral) before you treat 1 CR as the default ask.

12. 1 CR readiness scorecard (print)

Check before mass applying:

- ☐ 220+ LC with 120+ **medium** and 15+ **hard attempted**
- ☐ Intervals, Union-Find, Topo, Backtracking, Trie — each 3 problems blind
- ☐ Company-tagged set for your first 2 companies done twice
- ☐ 10 product HLDs on paper including Feed, Chat, Uber, Payment, YouTube, Drive, Typeahead
- ☐ 6 backend LLD/MC + 2 frontend MC timed
- ☐ CS pack: 10 rapid-fire without notes
- ☐ 10 STAR stories tagged to Amazon/Google/Meta
- ☐ Resume bullets have metrics; GitHub has architecture diagrams
- ☐ 1 full loop mock passed (2 DSA + 1 HLD + behavioral)
- ☐ You can explain TaskFlow + ShopLite for 20 minutes each

13. What was missing before this file (and is now required)

Gap	Why it blocked 1 CR	Fix
DSA was pattern-only	Product companies add follow-ups + company flavor	§4
HLD drifted to Kafka/K8s	Google/Meta grade product + API + bottleneck	§5 + Phase 7F rewrite
No frontend machine coding	Atlassian/Uber fail you	§6
No CS fundamentals pack	Microsoft/Amazon sneak these in	§7
One generic behavioral	Amazon LPs / Googleyness differ	§8
No take-home / HM / product sense	Real loops include them	§2, §8, §9
6 months only	L5 / 1 CR+ often needs more mock volume	Phase 9

Next: open [19-phases-678-easy-to-hard-complete.pdf](#) — Phases 6–8 now point at this file.

Second-pass gaps (OA, SQL drills, browser, a11y, GraphQL-lite, 1 Cr offer math, MERN verdict): [21-1cr-remaining-gaps-complete.pdf](#) .

Top 100 companies — every hiring stage: [22-top-100-recruitment-processes-complete.pdf](#) .